



CSE471: System Analysis and Design Project Report

Project Title: Project Manager Application

**Group No: 09, CSE471 Lab Section: 04,
Fall 2024**

Group No: 09, CSE471 Lab Section: 04, Fall 2024	
ID	Name
22101634	Rwittik Sarker
21301619	Inzamul Kabir Tanvir
21301236	Ahmad Habibullah

Submission Date: 07/01/2025
Table of Contents

1. System Request 3 Business need: 3 Business requirements: 3 Business value: 3 Special issues or constraints: 3
2. Functional Requirements 4 3. Technology (Framework, Languages) 5 4. Backend Development 5 5. User Interface Design 12 6. Frontend Development 16 7. User Manual 42 8. Performance and Network Analysis 46 9. Github Repo [Public] Link 49 10. Link of Deployed Project 49 11. Individual Contribution 50 12. References 51

1. System Request

Business need:

The Project Manager Application helps users efficiently track, organize, and manage their projects by providing tools for task assignment, progress monitoring, and deadline management in one centralized platform.

Business requirements:

Using Project Management Application, users will be able to better organize their projects and have them all in one place. The specific functionality that the system should have includes the following:

- Develop a secure and intuitive platform for efficient project and task management.
- Enable real-time progress tracking, dashboard visualizations, and task prioritization control.
- Provide role-based access control and administrative tools to ensure smooth system operations and user permissions management.

Business value:

We expect that the Project Manager Application will improve personal productivity and task management for individual users by providing an intuitive platform to organize projects, prioritize tasks, and track progress effectively. The application's user-friendly design and comprehensive features will attract individuals looking for a streamlined way to stay organized and meet deadlines.

Over time, we anticipate increased engagement as users leverage the platform to manage multiple projects efficiently, benefiting from features like customizable dashboards, progress tracking, and task prioritization. The application's scalability ensures it can adapt to future enhancements as user needs evolve, fostering long-term user satisfaction and retention.

Special issues or constraints:

- The development team views this as a strategic tool for enhancing individual productivity and task management. Keeping it simple and easy to use is essential, as these features will help attract more users in the growing project management market.

- Many potential users have expressed frustration with overly complex or expensive project management tools, emphasizing the need for an affordable, user-friendly solution. Failure to address these expectations could result in losing market relevance and user trust.

2. Functional Requirements

Module 1: Profile Management

1. Users can create an account by entering their name, email, and password. Users can securely log in to access their tasks and personalized features.
2. Users can view and update their **profile information**, such as name, email. **Change password** feature allows users to update their login credentials.
3. Admins can edit user roles and permissions to ensure proper access control.
4. Users can view and add projects.

Module 2: Task and Project Organization

1. Users can **create tasks** with details like title, description, due date, and priority level.
2. Tasks can be organized within **projects** to keep related tasks grouped together.
3. Users can mark tasks as “completed,” “in progress,” or “pending” to track progress.
4. Users can assign **priority levels** (e.g., high, medium, low) to focus on important tasks.

Module 3: Dashboard and Notifications

1. A **dashboard** shows an overview of tasks, project status, and progress metrics, including a project matrix that visualizes various informations of the project.
2. Users receive **notifications** for upcoming deadlines and completed tasks to stay updated.
3. **Search and filter** options allow users to locate projects by keywords.
4. Users can view their **activity history** for a summary of recent actions.

3. Technology (Framework, Languages)

Javascript, MERN stack(MongoDB, ExpressJs, ReactJs, NodeJs)

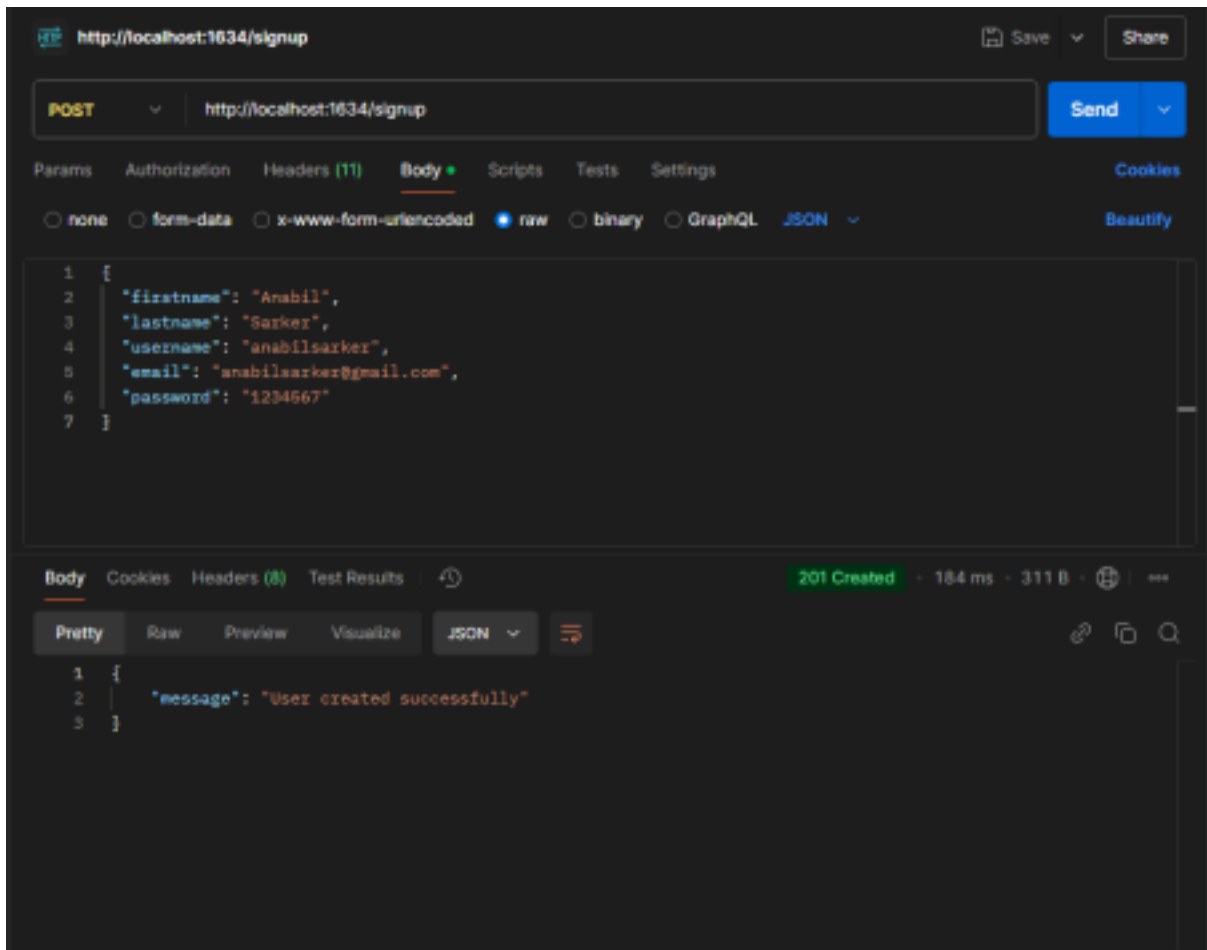
4. Backend Development

Attach at least 5 code snippets of Backend/API development with proper description. Also, attach corresponding screenshots of Postman API testing.

```
// Sign-Up API
app.post("/signup", async (req, res) => {
  const { firstname, lastname, username, email, password } =
    req.body;
  if (!firstname || !lastname || !username || !email || !password) {
    return res.status(400).json({ error: "Missing required fields" });
  }
  const userExists = await User.findOne({ email: email });
  if (userExists) {
    return res.status(400).json({ error: "Email already exists" });
  }
  const hashedPassword = await bcrypt.hash(password, 10); const
  newUser = new User({ firstname, lastname, username, email, password:
    hashedPassword });
```

```
res.status(201).json({ message: "User created successfully" });
});
```

This API allows new users to register by providing their first name, last name, username, email, and password. It validates that all required fields are present and checks whether the provided email already exists in the database. If the email is unique, the password is securely hashed using bcrypt, and a new user record is created and saved in the database. Upon successful registration, the API responds with a success message and a status code of 201. If any validation fails, appropriate error messages are returned with a 400 status code.



```
// Login API
app.post("/login", async (req, res) => {
  const { email, password } = req.body;
  if (!email || !password) {
    return res.status(400).json({ error: "Missing email or password" });
  }
  const user = await User.findOne({ email: email });
  if (!user) {
```

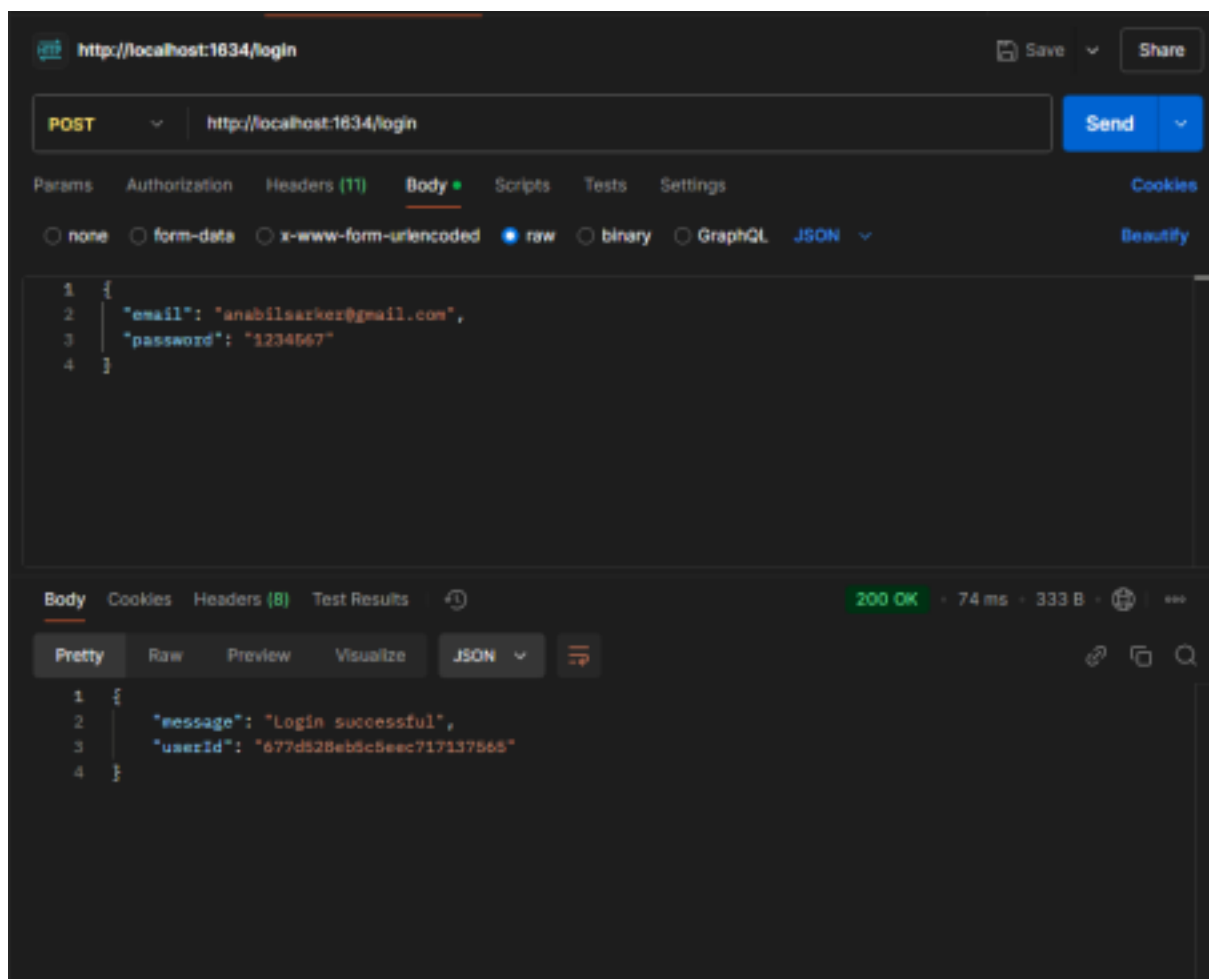
```
    return res.status(401).json({ error: "Invalid email or password" });
  }
```

```

    }
    const isPasswordValid = await bcrypt.compare(password,
user.password);
    if (!isPasswordValid) {
        return res.status(401).json({ error: "Invalid email or
password" });
    }
    res.json({ message: "Login successful", userId: user.id });
});

```

This API allows users to log in by providing their email and password. It first validates that both fields are present in the request. The system then checks if a user with the provided email exists in the database. If no such user is found or if the password provided is incorrect (validated using bcrypt), the API responds with an error and a status code of 401. Upon successful authentication, it returns a success message along with the user's unique ID.



7

```
// View Project API
```

```

app.get("/user/:id/projects", async (req, res) => {

    const userId = req.params.id;

```

```

    try {

        const user = await User.findById(userId).populate("projects");

        if (!user) {

            return res.status(404).json({ message: "User not found" });

        }

        const projects = user.projects || [];

        if (projects.length === 0) {

            return res.status(200).json({ message: "No Projects",
projects: projects });

        }

        res.status(200).json({ projects: projects });

    } catch (error) {

        console.error("Error fetching user projects:", error);

        res.status(500).json({ message: "Server error" });

    }

});

```

This API allows users to view all projects associated with their account by providing their unique user ID in the URL. It retrieves the user's data from the database and populates the related projects. If the user does not exist, the API responds with a 404 error. If no projects are found, a 200 status with a message "No Projects" is returned. In the case of existing projects, the API returns them in the

response with a 200 status. If any server error occurs during this process, a 500 error message is returned.

http://localhost:1634/user/6772878919d78c35a34cf25c/projects

GET http://localhost:1634/user/6772878919d78c35a34cf25c/projects Send

Params Authorization Headers (11) Body • Scripts Tests Settings Cookies

Query Params

Key	Value	Description
Key	Value	Description

Body Cookies Headers (8) Test Results 200 OK · 18 ms · 567 B

Pretty Raw Preview Visualize JSON

```

1 {
2   "projects": [
3     {
4       "_id": "6772f7a505d586f793c230c5",
5       "name": "cars",
6       "tasks": [
7         "677b08cc8f887010735b0a5a",
8         "677b0b16e9014b23d0272130",
9         "677b0c456a5bede992685bcb"
10      ],
11      "__v": 3
12    },
13    {
14      "_id": "6772f0420bd506f793c230cc",
15      "name": "cars",
16      "tasks": [],
17      "__v": 0
18    },
19    {
20      "_id": "6772f8a90bd506f793c230d0",

```

// Add Project API

```

app.post("/user/:id/projects/add", async (req, res) => {

  const { name } = req.body;

  const userId = req.params.id;

  if (!name) {

    return res.status(400).json({ error: "Missing required fields"
  });

}

```

9

```

try {

  const newProject = new Project({ name });

  const savedProject = await newProject.save();

```

```

    const user = await User.findByIdAndUpdate(
        userId,
        { $push: { projects: savedProject._id } },
        { new: true }
    );

    if (!user) {
        return res.status(404).json({ error: "User not found" });
    }

    res.status(201).json({ message: "Project added successfully",
project: savedProject });

    } catch (error) {

        console.error("Error adding project:", error);

        res.status(500).json({ error: "An error occurred while adding
the project" });

    }

});

```

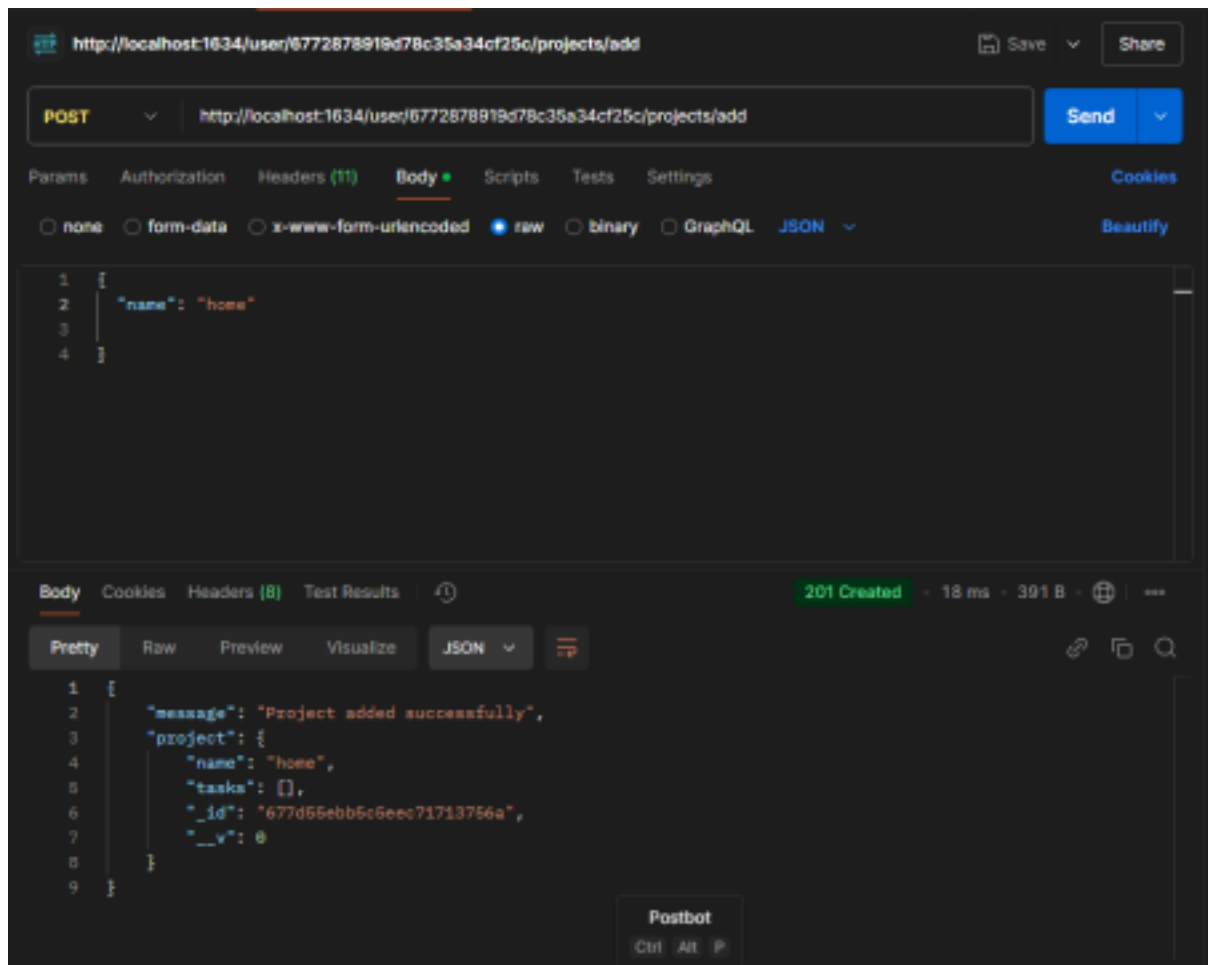
This API allows users to add a new project to their account by providing the project name and their user ID in the URL. It validates that the project name is provided and creates a new project in the database. The newly created project's ID is then linked to the user by updating their account with the project reference.

- If the project name is missing, the API responds with a 400 error.

10

- If the user ID is invalid or the user does not exist, a 404 error is returned.
- Upon successful creation and association, the API responds with a 201 status and details of the created project.

If a server error occurs, the API responds with a 500 error and an appropriate message.



```
// Admin View API
app.get("/admin/users", async (req, res) => {
  try {
    const users = await User.find({})
      .populate("projects", "name")
      .select("firstname lastname username email role
permissions activityHistory projects");
    res.status(200).json(users);
  } catch (error) {
    console.error("Error fetching users:", error);
    res.status(500).json({ error: "An error occurred while fetching
users" });
  }
});
```

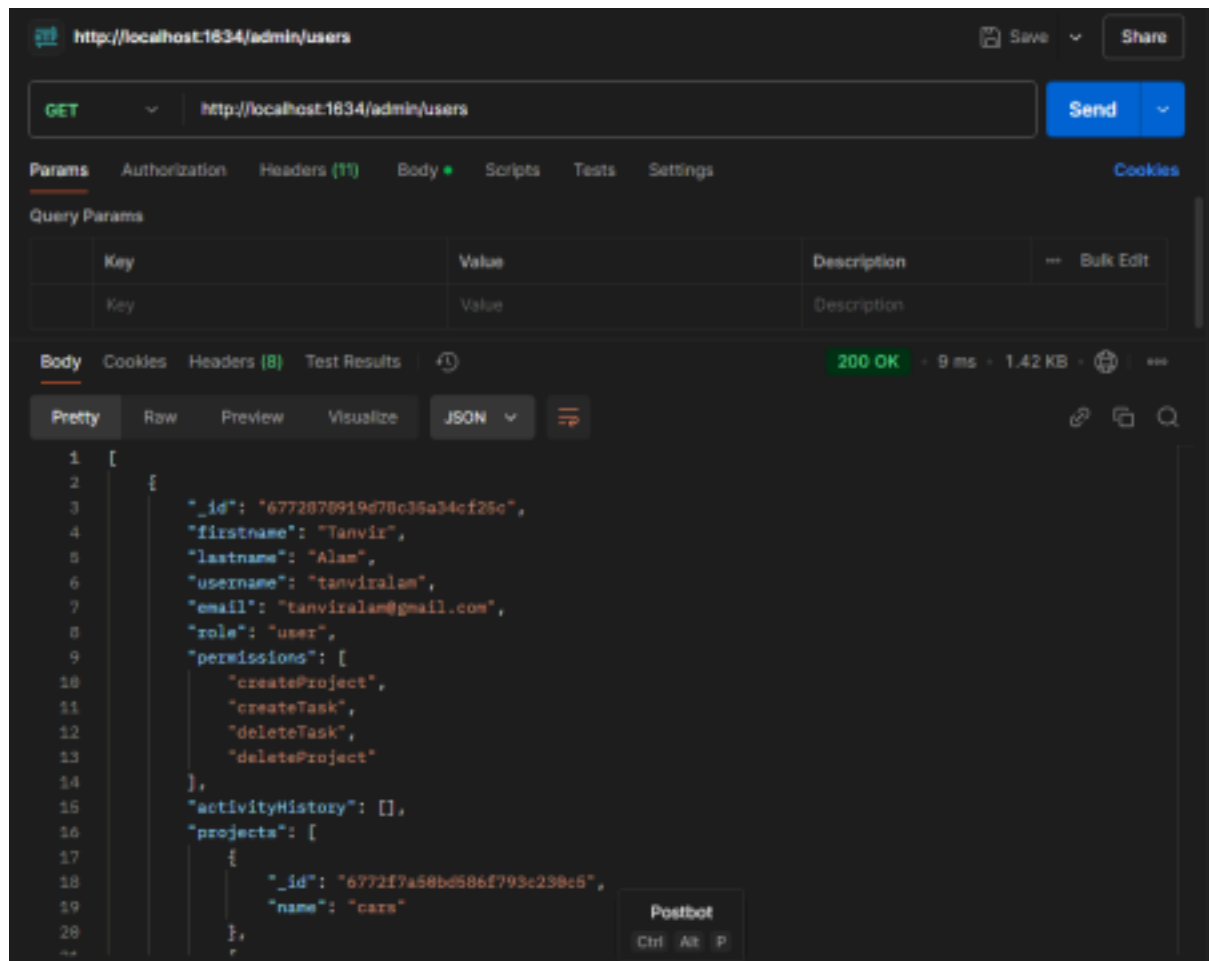
11

This API endpoint allows administrators to view a detailed list of all registered users in the system. It retrieves user data from the database, including their first name, last name, username, email, role, permissions, activity history, and associated projects. Each project is populated to display only its name.

- If the operation is successful, the API responds with a 200 status and the list of users.

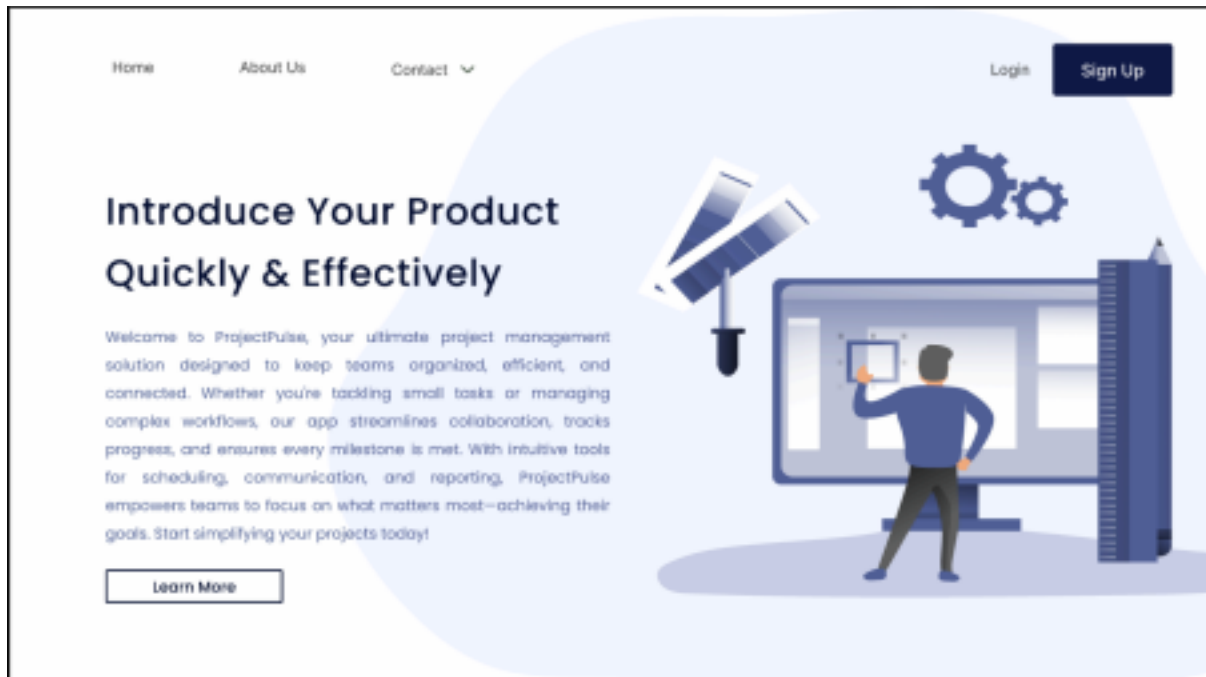
- If an error occurs while fetching the data, the API returns a 500 error along with an appropriate error message.

This endpoint is intended for administrative use to monitor user details and project associations.



5. User Interface Design

Landing Page:



Sign up Page:

A sign-up form with a dark background and a white grid pattern. The form is a white rounded rectangle containing a toggle for "Sign up" and "Log in". Below the toggle, the heading "Sign up" is centered. The form includes input fields for "First name", "Last name", and "Email address", followed by a "Sign up" button.

13

Login Page:

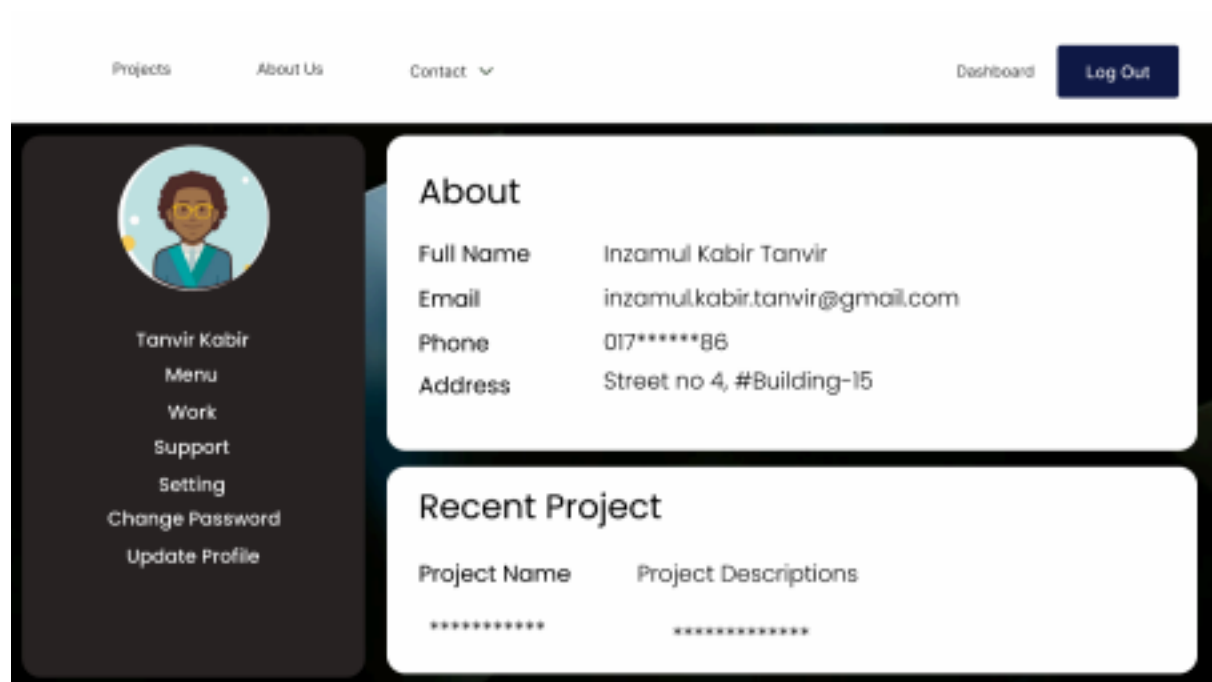


Project Page:

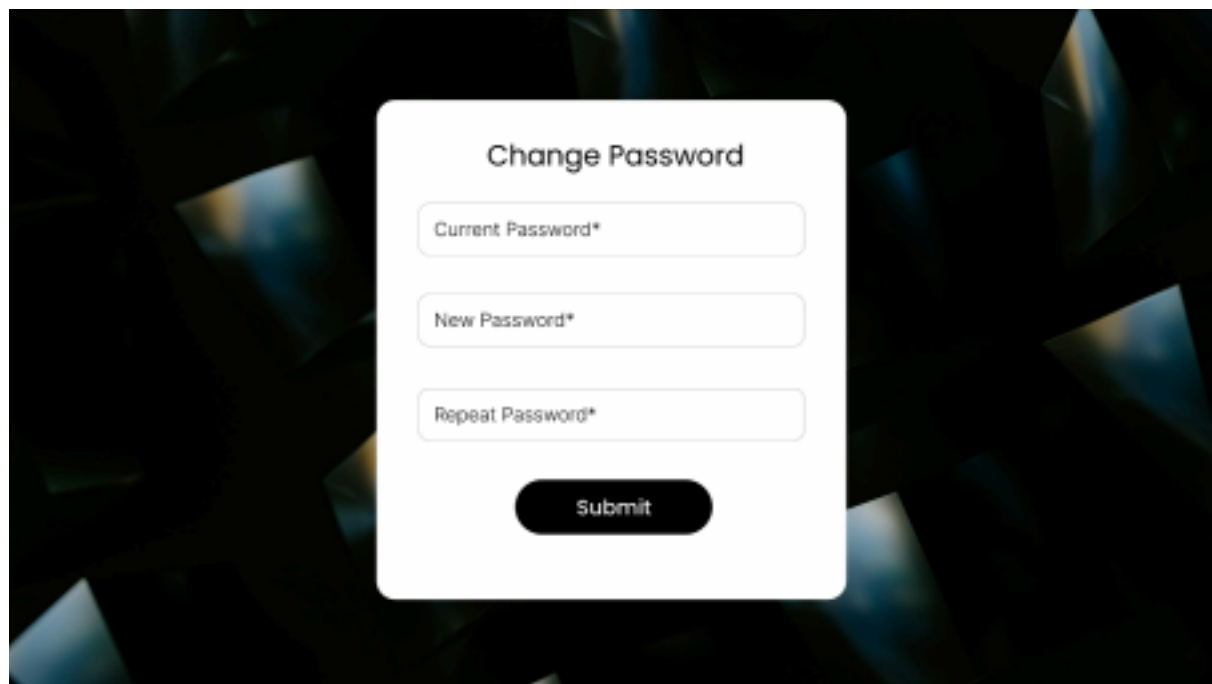
Projects	Profile	About Us	Contact ▾	Dashboard	Log Out
Show 10 entries	Search...	+ Add Project			
ID	Project	Date	Status	Action	
#20462	 Ravi	13/05/2022	Completed	 	
#20463	 Tanvir	13/05/2022	Pending	 	
#20464	 Shovan	13/05/2022	Canceled	 	
Previous 1 2 3 Next					

14

Profile Page:

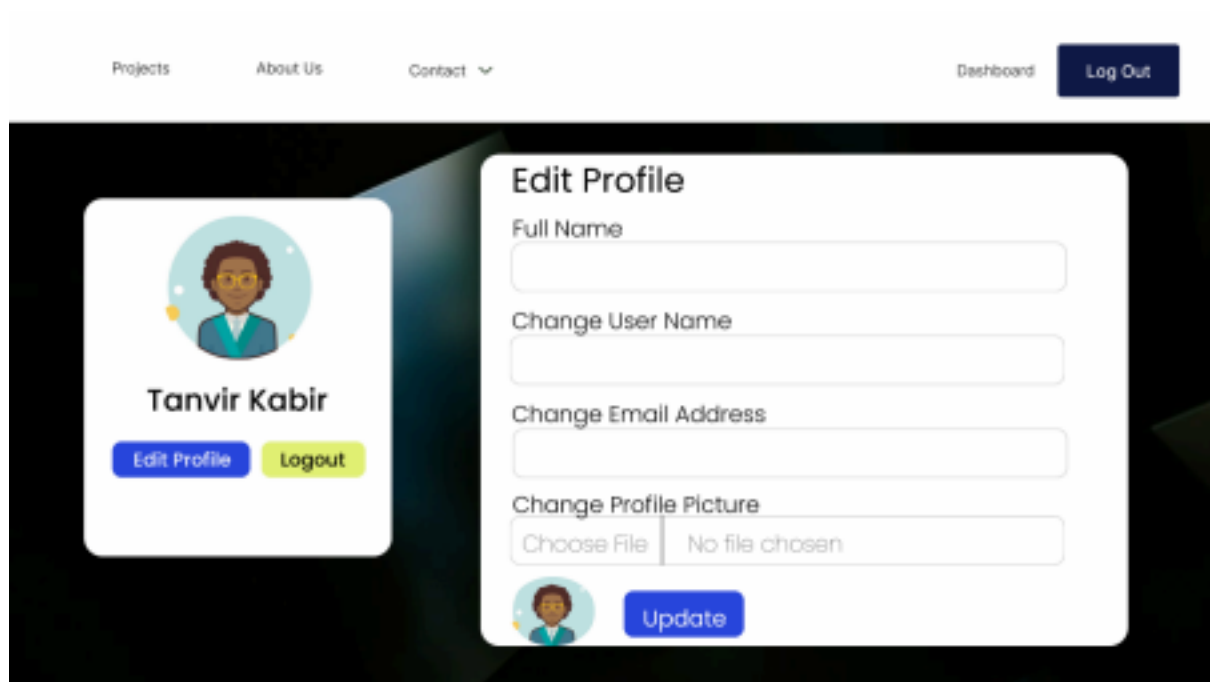


Change Password:



The image shows a 'Change Password' form centered on a dark background with abstract blue and yellow geometric shapes. The form is a white rounded rectangle with the title 'Change Password' at the top. It contains three input fields, each with a label and an asterisk indicating it is required: 'Current Password*', 'New Password*', and 'Repeat Password*'. Below these fields is a black rounded button with the text 'Submit' in white.

Edit Profile:



Link: <https://www.figma.com/proto/2snn6pN4LHlyJKfTc4vALO/Project-Manager-App?node-id=1-2&starting-point-node-id=1%3A2&t=qe4nAXeCywOzguai-1>

6. Frontend Development

Attach at least 5 frontend code snippets with proper description.

1. This React component renders the landing page for **ProjectPlus**. It features:
 - **Navbar:** Includes links to "About Us" and "Contact" popups, along with Login and Sign-Up buttons for navigation.
 - **Main Content:** Introduces the app with a headline, description, and placeholder illustration.
 - **Pop Ups:** Displays "About Us" and "Contact Us" information in toggleable popups.

It uses React Router's useHistory for navigation and external CSS for styling.

```
import React, { useState } from "react";

import { useHistory } from "react-router-dom";
```

16

```
import "../LandingPage.css";

const LandingPage = () => {

  const [showAboutPopup, setShowAboutPopup] = useState(false);

  const [showContactPopup, setShowContactPopup] = useState(false);

  const history = useHistory();
```

```

const togglePopup = (popupType) => {

  if (popupType === "about") {

    setShowAboutPopup(!showAboutPopup);

  } else if (popupType === "contact") {

    setShowContactPopup(!showContactPopup);

  }

};

return (

  <div className="landing-page">

    <header className="navbar">

      <div className="navbar-title">ProjectPlus</div>

      <nav>

        <a href="#about" onClick={() => togglePopup("about")}>

          About Us

        </a>

        <a href="#contact" onClick={() => togglePopup("contact")}>

          Contact

        </a>

      </nav>

      <div className="navbar-buttons">

        <button onClick={() => history.push("/login")}>Login</button>

        <button className="signup-btn" onClick={() =>
history.push("/signup")}>

          Sign Up

        </button>

```

```

    </div>

</header>

<main className="main-content">

    <div className="main-text">

        <h1>

            Introduce Your Product <br />

            Quickly & Effectively

        </h1>

        <p>

            Welcome to ProjectPlus, your ultimate project management
solution

            designed to keep teams organized, efficient, and
connected. Whether

            you're tackling small tasks or managing complex
workflows, our app

            streamlines collaboration, tracks progress, and ensures
every

            milestone is met. With intuitive tools for scheduling,

18

            communication, and reporting, ProjectPlus empowers teams
to focus

            on what matters most—achieving their goals. Start
simplifying your

            projects today!

        </p>

        <button>Learn More</button>

```

```

    </div>

    <div className="illustration">

        <img
src={require('../resources/illustration-placeholder.png')}
alt="illustration"/>

    </div>

</main>

{showAboutPopup && (

    <div className="popup about-popup">

        <div className="popup-content">

            <h2>About Us</h2>

            <p>

                ProjectPlus is dedicated to streamlining project
management for

                teams worldwide. We focus on collaboration and
efficiency.

            </p>

            <button onClick={ () =>
togglePopup("about") }>Close</button>

        </div>

    </div>

    19

</div>

)}

{showContactPopup && (

    <div className="popup contact-popup">

        <div className="popup-content">

```

```

        <h2>Contact Us</h2>

        <p>Email: support@projectplus.com</p>

        <p>Phone: +880123456789</p>

        <button onClick={ () =>
togglePopup("contact") }>Close</button>

    </div>

</div>

    ) }

</div>

    );

};

export default LandingPage;

```

2. This React component handles **Sign Up** and **Log In** functionality:

- **Dynamic Form:** The form dynamically adapts based on the URL (/signup or /login) to show relevant fields.
- **User Input Handling:** Tracks user input with useState and updates form data.
- **Form Submission:** Sends a POST request to the appropriate endpoint (/signup or /login).
- **Navigation:** Redirects users based on their role (admin or standard user) using useHistory.
- **Styling:** Includes custom CSS and a background image.

This component ensures seamless authentication for both new and returning users.

```

import React, { useState } from "react";

import { useHistory, useLocation } from "react-router-dom";

import "../AuthPage.css";

const AuthPage = () => {

```

```

const location = useLocation();

const history = useHistory();

const isSignUp = location.pathname === "/signup";

const [formData, setFormData] = useState({

  firstName: "",

  lastName: "",

  username: "",

  email: "",

  password: "",

});

const handleChange = (e) => {

  const { name, value } = e.target;

  setFormData((prev) => ({ ...prev, [name]: value }));

};

const handleSubmit = async (e) => {

  e.preventDefault();

  var endpoint = "";

  var payload = [];

  if (isSignUp) {

    endpoint = "http://localhost:1634/signup"

```

```

    }

    else {

        endpoint = "http://localhost:1634/login";

    }

    if (isSignUp) {

        payload = {

            firstname: formData.firstName,

            lastname: formData.lastName,

            username: formData.username,

            email: formData.email,

            password: formData.password,

        };

        console.log(payload);

    }

    else {

        payload = {

            email: formData.email,

            password: formData.password,

        };

        22

    };

```

```

try {

    const req = new Request(

        endpoint, {

```

```

    method: "POST",

    headers: {

        "Content-Type": "application/json",

    },

    body: JSON.stringify(payload),

});

console.log(req);

const response = await fetch(req);

console.log(response);

if (response.ok) {

    const data = await response.json();

    console.log(`${isSignUp ? "Sign up" : "Log in"} successful`,
data);

    if (formData.email === "admin@example.com") {

        history.push("/adminview");

    }

    else{

        history.push("/projects", { userId: data.userId });

    }

} else {

```

23

```

const errorData = await response.json();

console.error("Error:", errorData.error);

alert(errorData.error );

}

} catch (error) {

```

```

        console.error("Error:", error);

        console.log("this is error: " + error);

        alert("Network error. Please try again."); }

};

return (

    <div

        className="auth-page"

        style={{

            backgroundImage:
`url(${require("../resources/login-
            signup.webp")} ` , }}

    >

    <div className="auth-container">

        <div className="toggle-buttons">

            <button

                className={isSignUp ? "active" : ""}

                onClick={() => history.push("/signup")}

            >


```

24

```

        Sign up

    </button>

    <button

        className={!isSignUp ? "active" : ""}

        onClick={() => history.push("/login")} >

        Log in


```

```

    </button>

</div>

<div className="form-container">

  <h2>{isSignUp ? "Sign up" : "Log in"}</h2>

  <form onSubmit={handleSubmit}>

    {isSignUp && (

      <div className="form-group">

        <input

          type="text"

          name="firstName"

          placeholder="First name"

          value={formData.firstName}

          onChange={handleChange}

          required

        />

        <input

          type="text"

          name="lastName"

          placeholder="Last name"

          value={formData.lastName}

          onChange={handleChange}

          required

        />

      </div>
    )}
  </form>
</div>

```

```

    ) }

    {isSignUp && (

      <input

        type="text"

        name="username"

        placeholder="User name"

        value={formData.username}

        onChange={handleChange}

        required

      />

    ) }

```

```

<input

  type="email"

  name="email"

  placeholder="Email address"

  value={formData.email}

  onChange={handleChange}

  required


```

26

```

/>

<div className="password-container">

  <input

    type="password"

    name="password"

    placeholder="Your password"

    value={formData.password}


```

```

        onChange={handleChange}

        required

      />

</div>

{!isSignUp && (

  <a href="#" className="forgot-password">

    Forget your password

  </a>

)}

<button className="form-submit" type="submit">

  {isSignUp ? "Sign up" : "Log in"}

</button>

</form>

</div>

</div>

</div>

);

};

```

27

```
export default AuthPage;
```

3. This React component displays a user's projects with features to:

- Fetch and list projects for a specific userId.
- Search projects by name using a filter.
- Navigate to project-specific tasks, profile, or add new projects.
- Handle empty or error states with appropriate messages.

It provides a clean, interactive interface for project management. `import React,`

```

{ useEffect, useState } from "react"; import { useHistory,
useLocation } from "react-router-dom"; import
"./global.css";

import "./ProjectsPage.css";

```

```

const ProjectsPage = () => {

  const [projects, setProjects] = useState([]);

  const [message, setMessage] = useState("");

  const [searchTerm, setSearchTerm] = useState("");

  const history = useHistory();

  const location = useLocation();

  const userId = location.state.userId;

  useEffect(() => {

    const fetchProjects = async () => {

      try {

```

28

```

      const response = await
fetch(`http://localhost:1634/user/${userId}/projects`);

      const data = await response.json();

      if (response.ok) {

        setProjects(data.projects);

        if (data.projects.length === 0) {

          setMessage("No Projects");

        }

```

```

        } else {

            setMessage(data.message || "Failed to fetch
projects");

        }

    } catch (error) {

        console.error("Error fetching projects:", error);

        setMessage("Network error. Please try again.");

    }

};

if (userId) {

    fetchProjects();

} else {

    setMessage("User not authenticated.");

}

}, [userId]);

```

29

```

const handleProjectClick = (projectId, userId) => {

    history.push("/tasks", { projectId: projectId , userID:
userId});

};

const filteredProjects = projects.filter(project =>

    project.name.toLowerCase().includes(searchTerm.toLowerCase())

);

```

```

return (

  <div className="projects-page">

    <header className="navbar">

      <div className="navbar-links">

        <a onClick={ () => history.push("/profile", {
userId: userId })}>Profile</a>

        <a onClick={ () => history.push("/projects", {
userId: userId })}>Projects</a>

        <a href="#contact">Contact <span>&#9660;</span></a>

      </div>

      <div className="navbar-actions">

        <a href="#dashboard">Dashboard</a>

        <button className="logout-btn" onClick={ () =>
history.push("/logout")}>Log Out</button>

      </div>

    </header>

    <div className="projects-header">

      <div className="search-box">

        30

        <input

          type="text"

          placeholder="Search projects..."

          value={searchTerm}

          onChange={ (e) => setSearchTerm(e.target.value)}

        />

      </div>

      <button className="add-project-btn" onClick={ () =>
history.push("/addprojectpage", { userId: userId })}>+ Add
Project</button>

```

```

</div>

<table className="projects-table">

  <thead>

    <tr>

      <th>#</th>

      <th>Project Name</th>

    </tr>

  </thead>

  <tbody>

    {filteredProjects.length > 0 ? (

      filteredProjects.map((project, index) => (

        <tr key={project._id} onClick={() =>
handleProjectClick(project._id)}>

          <td>{index + 1}</td>

          <td>{project.name}</td>

        </tr>

      ))

    ) : (

      <tr>

        <td colspan="2">{message || "No Projects
Found"}</td>

      </tr>

    ) }

  </tbody>

</table>

```

```

        </div>

    );

};

export default ProjectsPage;

```

4. This React component displays a user's profile with the following features:
- **Fetch and Display Profile Data:** Retrieves and shows user details based on a specific `userId`.
 - **Edit Profile:** Allows users to edit their name and email, with changes saved via a PUT request.
 - **Navigation:** Provides links for navigating to different sections such as projects, password change, and logout.
 - **Error and Loading States:** Handles errors and loading states gracefully, displaying appropriate messages when necessary.

The component ensures an interactive and customizable profile management experience.

```

import { useHistory, useLocation } from "react-router-dom";

import { useState, useEffect } from "react";

```

32

```

import "./ProfilePage.css";

import "./global.css";

const ProfilePage = () => {

    const location = useLocation();

    const userId = location.state.userId;

    const history = useHistory();

    const [user, setUser] = useState(null);

    const [loading, setLoading] = useState(true);

```

```

    const [error, setError] = useState(null);

    const [isEditing, setIsEditing] = useState({ name: false, email:
false });

    const [updatedName, setUpdatedName] = useState("");

    const [updatedEmail, setUpdatedEmail] = useState("");

    useEffect(() => {

        const fetchUserProfile = async () => {

            try {

                const response = await
fetch(`http://localhost:1634/user/${userId}`);

                if (!response.ok) {

                    throw new Error("Failed to fetch user profile");

                }

                const data = await response.json();

                setUser(data);

                33
                setUpdatedName(`${data.firstname} ${data.lastname}`);

                setUpdatedEmail(data.email);

            } catch (err) {

                setError(err.message);

            } finally {

                setLoading(false);

            }

        };

        if (userId) {

```

```

        fetchUserProfile();

    } else {

        setError("No user ID provided");

        setLoading(false);

    }

}, [userId]);

const handleSave = async () => {

    const [firstname, lastname] = updatedName.split(" ");

    try {

        const response = await
fetch(`http://localhost:1634/user/${userId}`, {

            method: "PUT",

            headers: { "Content-Type": "application/json" },

34
            body: JSON.stringify({ firstname, lastname, email:
updatedEmail }),

        });

        if (!response.ok) {

            throw new Error("Failed to update profile");

        }

        const updatedUser = await response.json();

        setUser(updatedUser);

        setIsEditing({ name: false, email: false });

```

```

    } catch (err) {

        setError(err.message);

    }

};

    if (loading) return <div className="loading">Loading
profile...</div>;

    if (error) return <div className="error">Error: {error}</div>;

    return (

        <div className="profile-page">

            <header className="navbar">

                <div className="navbar-links">

                    <a onClick={ () => history.push("/profile", {
userId: userId })}>Profile</a>

                    35
                    <a onClick={ () => history.push("/projects", {
userId: userId })}>Projects</a>

                    <a href="#contact">Contact <span>&#9660;</span></a>

                </div>

                <div className="navbar-actions">

                    <a href="#dashboard">Dashboard</a>

                    <button className="logout-btn" onClick={ () =>
history.push("/logout")}>Log Out</button>

                </div>

            </header>

            <div className="profile-container">

```

```

<aside className="sidebar">

  <ul className="sidebar-links">

    <li>{user.firstname} {user.lastname}</li>

    <li>Menu</li>

    <li>Work</li>

    <li>Support</li>

    <li>Setting</li>

    <a onClick={() =>
history.push("/changepassword", { userId: userId })}>Change
Password</a>

  </ul>

</aside>

<main className="main-content1">

  <div className="info-card about-card">

    36
    <h2>About</h2>

    <p>

      <strong>Full Name:</strong>

      {isEditing.name ? (

        <input

          type="text"

          value={updatedName}

          onChange={(e) =>
setUpdatedName(e.target.value)}

        />

      ) : (

        <span onClick={() => setIsEditing({

```

```

...isEditing, name: true }}}>

        {user.firstname} {user.lastname}

    </span>

    )}

</p>

<p>

    <strong>Email:</strong>

    {isEditing.email ? (

        <input

            type="email"

            value={updatedEmail}

            onChange={ (e) =>

setUpdatedEmail (e.target.value) }

        />

        37

    ) : (

        <span onClick={() => setIsEditing({

...isEditing, email: true }}}>

            {user.email}

        </span>

    )}

</p>

{isEditing.name || isEditing.email ? (

    <button onClick={handleSave}>Save</button>

) : null}

<p><strong>Username:</strong>

{user.username}</p>

```

```

        <p><strong>Role:</strong> {user.role}</p>

      </div>

    </main>

  </div>

</div>

);

};

export default ProfilePage;

```

5. This React component provides an interface for editing a project with the following features:

- **Edit Project Information:** Displays a form with fields to update the project ID, description, date, and status.
- **Save Changes:** A "Save" button allows users to submit the updated project information.

38

- **Navigation:** Includes links to navigate to projects, about, and contact pages, as well as a logout button.
- **Interactive Form:** The form includes input fields with placeholders to guide the user in editing project details.

The component offers a clean and functional interface for updating project data.

```

import './AddProjectPage.css';

import './EditProjectPage.css';

import './global.css';

const EditProjectPage = () => {

  return (

    <div className="add-project-page">

```

```

<header className="navbar">

  <div className="navbar-links">

    <a href="#projects">Projects</a>

    <a href="#about">About Us</a>

    <a href="#contact">Contact <span>&#9660;</span></a>

  </div>

  <div className="navbar-actions">

    <a href="#dashboard">Dashboard</a>

    <button className="logout-btn" onClick={() =>
history.push("/logout")}>Log Out</button>

  </div>

</header>

<div className="header-bar">

  <h3 className="add-project-btn"> Edit Project</h3>

</div>

```

39

```

<div className="form-container">

  <form className="add-project-form">

    <div className="form-group">

      <label htmlFor="projectId">

        <span className="icon"></span> Project ID

      </label>

      <input type="text" id="projectId"
placeholder="Enter Project ID" />

    </div>

    <div className="form-group">

```

Description

```
<label htmlFor="projectDescription">
    <span className="icon"></span> Project
Description

</label>

<input
    type="text"
    id="projectDescription"
    placeholder="Enter Project Description"
/>
</div>
```

```
<div className="form-group">
    <label htmlFor="date">
        <span className="icon"></span> Date
        40
    </label>
    <input type="date" id="date" />
</div>
```

```
<div className="form-group">
    <label htmlFor="status">
        <span className="icon"></span> Status
    </label>
    <input type="text" id="status"
placeholder="Enter Status" />
</div>
```

```

        <button type="submit"
className="save-btn">Save</button>

    </form>

</div>

</div>

);

};

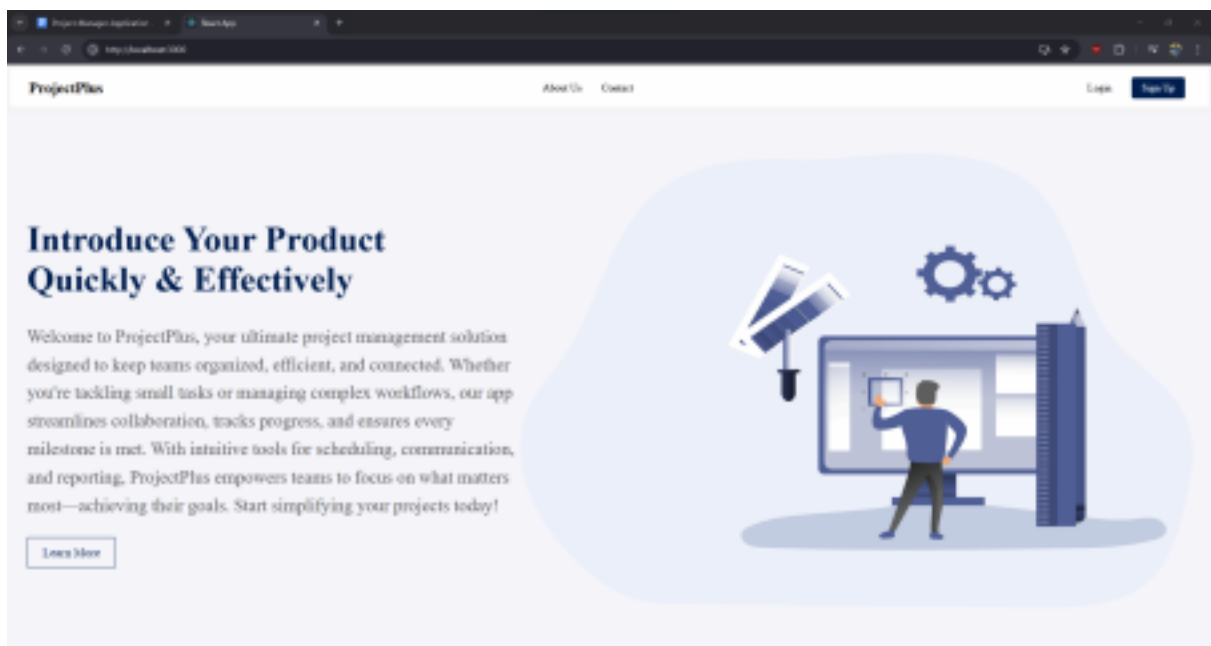
```

```
export default EditProjectPage;
```

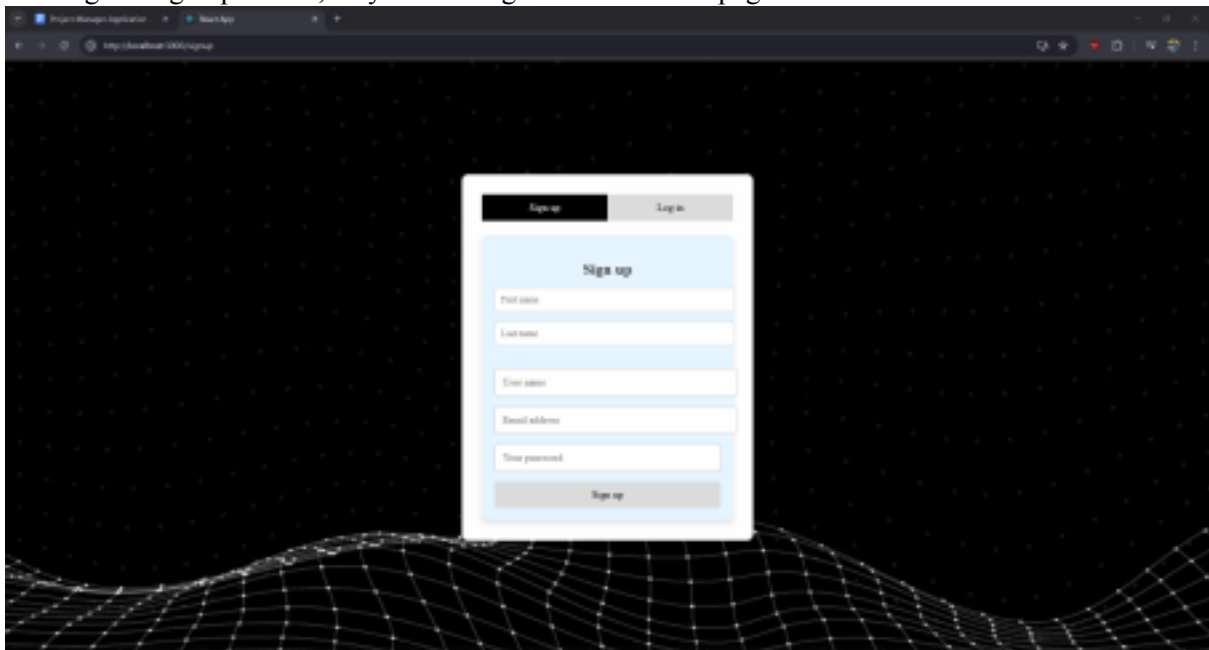
7. User Manual

How a new user will navigate or use the system with screenshots.

1. Upon entering the app, the user ends up in the Landing page. By clicking the About Us and Contact, users can view information about the developers and how to contact them, upon clicking login or signup, users will navigate to the login or signup page.



2. In the sign up page users will be able to input their information to create an account. And by clicking the sign up button, they'll be navigated to the home page.



42

3. In the login page, users will be able to login to their account and by clicking login, they'll be navigated to the home page.

4. In the home page, users can view all the projects they have added on their profile. By clicking on Profile, users will be able to navigate to their Profile page, by clicking Projects, they can come back to the home page. Users can also search projects by name, they can add projects by clicking on add project, and are able to logout by clicking on the logout button of the navbar. By clicking on a project, they can navigate to the tasks page.

5. In the profile page, users can view their profile information, they can change their name and email address by clicking on name and email address field. By clicking on Change Password,

43

the user can navigate to the change password page.

6. In the change password page, users are able to change their password by verifying old password and typing the new password twice.

7. In the add project page, users can add a new project by typing the name and clicking on save button.

8. In the tasks page, users can view the specific tasks of a project with all the parameters. Users can filter the tasks based on priority, by clicking on the Status field, they're able to change the Status to either "Pending", "Ongoing" or "Completed", by clicking on the add task button, they can navigate to the the add task page.

9. In the add task page, users can add a new task to their project by inputting the task title, description, priority level and due date. By clicking on the Save task button, the task is saved

45

under the project.

8. Performance and Network Analysis

Generate Performance and Network analysis report using lighthouse DevTool. Add screenshots of the report. Also, attach a screenshot of your system in any mobile viewport. **The system UI must be responsive.**

Performance:

Lighthouse:

46

NetworkAnalysis:

47

48

System UI responsiveness:

9. Github Repo [Public] Link:

<https://github.com/RwittikSarker/Project-Manager-App.git>

10. Link of Deployed Project:

<https://project-manager-app-mf6j.vercel.app/>

11. Individual Contribution

Group member - 01	
Name: Rwittik Sarker	Student ID: 22101634
Functional Requirements which are developed by this member:	
1. Users can create an account by entering their name, email, and password. Users can securely log in to access their tasks and personalized features.	
2. Users can view and update their profile information, such as name, email. Change password feature allows users to update their login credentials.	
3. Admins can edit user roles and permissions to ensure proper access control.	
4. Users can view and add projects.	
5. Users can create tasks with details like title, description, due date, and priority level.	
6. Tasks can be organized within projects to keep related tasks grouped together.	
7. Users can mark tasks as “completed,” “in progress,” or “pending” to track progress.	
8. Users can assign priority levels (e.g., high, medium, low) to focus on important tasks.	
9. Search and filter options allow users to locate projects by keywords.	

Group member - 02	
Name: Inzamul Kabir Tanvir	Student ID: 21301619
Functional Requirements which are developed by this member:	
1. Users can create an account by entering their name, email, and password. Users can securely log in to access their tasks and personalized features	
2. Users can view and update their profile information, such as name, email. Change password feature allows users to update their login credentials.	
3. Frontend	
4. Backend	

Group member - 03	
Name: Ahmad Habibullah	Student ID: 21301236
Functional Requirements which are developed by this member:	

1. Update Module 3: Updated Dashboard API, Try to add Notifications API, and Tasks API for better functionality.
--

2. Added a Task Notification Scheduler for alerts on upcoming deadlines. Push to the GitHub branch.
3. Frontend CSS
4. Backend

12. References